

CS6008 — Network Security

Assignment 1

Submitted by

Ramachandran S (23B1052)
Keyaan KR (23B0977)

September 4, 2026

1 Test Environment

1.1 Implementation summary

The application is written in **C++** (POSIX sockets) and uses OpenSSL’s *low-level* primitives only — `<openssl/bn.h>` for big-integer arithmetic, `<openssl/evp.h>` for AES-GCM and RSA signatures, and `<openssl/x509.h>` for certificate handling. In keeping with the brief, the TLS/SSL layer (`<openssl/ssl.h>`) and the packaged Diffie–Hellman API (`<openssl/dh.h>`) are *never* used: the key exchange, the encryption framing, the handshake and all certificate checks are our own code.

Two modules are shared, unchanged, by every phase: `common.cc` implements the wire framing, and `crypto.cc` (introduced in Phase 2) implements the cryptographic building blocks. Each phase directory is self-contained and independently buildable via its `Makefile`, exactly as required by the submission guidelines.

1.2 VM topology

The assignment was demonstrated across separate VirtualBox virtual machines on a common 10.176.76.0/24 network (not solely on loopback), so that the Wireshark captures and the man-in-the-middle demonstration reflect a real network deployment. The roles are shown in Figure 1 and Table 1.

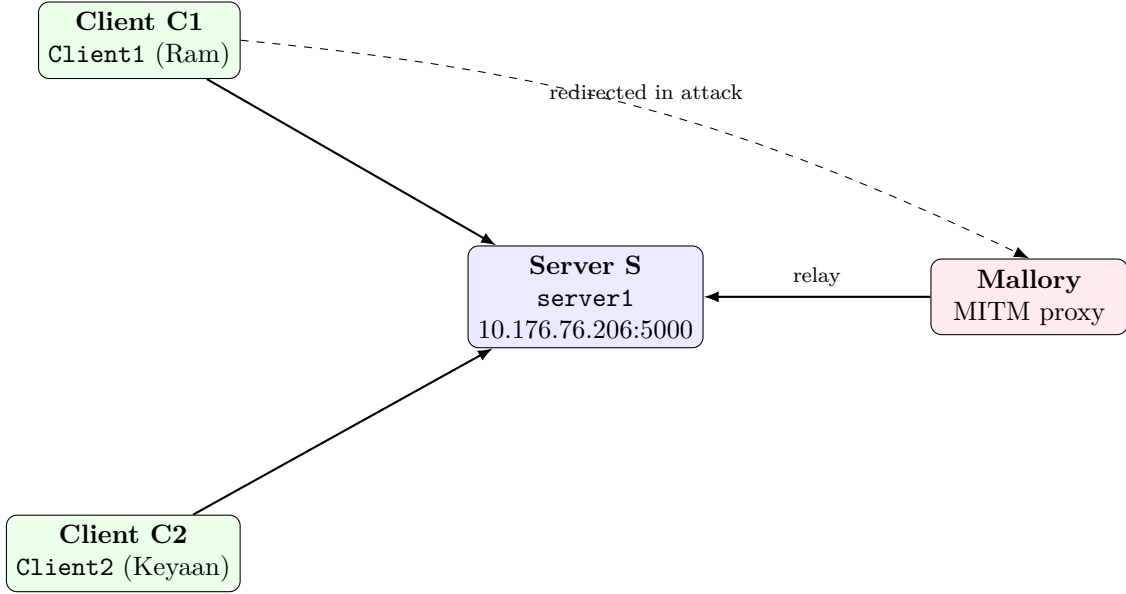


Figure 1: VM topology

Role	Hostname	Address (observed)	Port
Server S	server1	10.176.76.206	5000
Client C_1	Client1	10.176.76.207	—
Client C_2	Client2	10.176.76.207	—
Attacker (Mallory)	Mallory	10.176.76.206	5000 / 5001

Table 1: VM roles and addresses

1.3 Communication design

Message framing. TCP is a byte stream, so the application defines a message boundary explicitly: every payload is prefixed with a 4-byte big-endian length, and the receiver reads exactly that many bytes. A message is therefore “complete” once its advertised length has been fully read (`recv_frame` in `common.cc`). The maximum accepted frame is 64 KiB. This same framing is reused unchanged in every phase; from Phase 2 on, the payload is an AES-GCM ciphertext blob rather than plaintext.

Application protocol. Inside each frame is a simple text line. The client sends `LOGIN <name>`, `MSG <target> <text>`, `WHO` or `QUIT`; the server replies with `OK`, `MSG <from> <text>`, `WHO <csv>` or `ERR`. The server routes purely by username, which is what allows the relay to stay oblivious to the end-to-end layer added later. If more than two clients were to connect, the server simply keeps each in a table keyed by username and routes each `MSG` to the named recipient.

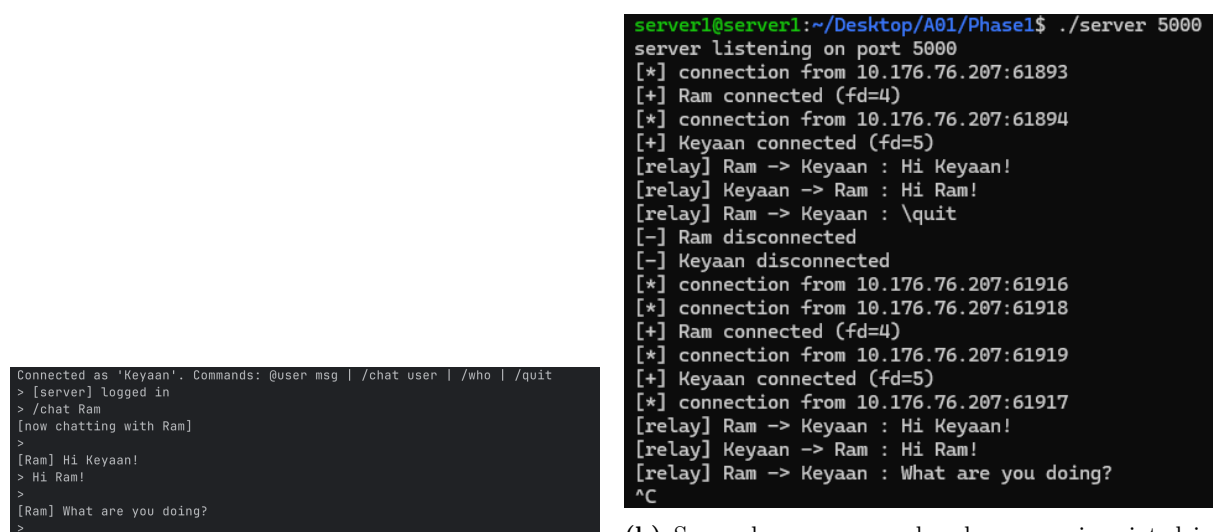
Fingerprints. Wherever a shared secret is established, both sides print a *fingerprint* — the first 8 bytes of `SHA-256(secret)` in hex. The raw secret and the derived key are never printed. Equal fingerprints on the two ends are strong evidence that they agree on the key.

2 Phase 1 - Baseline Chat (No Security)

2.1 Implementation

The server (`server.cc`) accepts each client on its own `std::thread` and keeps a mutex-protected table of `{fd, username}`. On receiving `MSG <target> <text>` it looks up the target and forwards `MSG <sender> <text>`. Crucially, it also prints `[relay] <from> -> <to> : <text>` for every message, demonstrating that at this stage the server has full visibility of all content. The client runs a receiver thread (printing incoming messages) alongside the main input loop that parses the command interface.

2.2 Verification and evidence



The figure consists of two terminal screenshots. The left screenshot, labeled (a), shows a client named 'Keyaan' interacting with a server. The client has logged in and is now chatting with a user named 'Ram'. The client sends the message 'Hi Keyaan!' and receives the response 'Hi Ram!'. The client then sends 'Hi Ram!' and receives 'What are you doing?'. The right screenshot, labeled (b), shows the server's log output. It displays the server listening on port 5000, followed by connection logs for two clients (Ram and Keyaan). The log shows the relay of messages between them in plaintext, such as '[relay] Ram -> Keyaan : Hi Keyaan!' and '[relay] Keyaan -> Ram : Hi Ram!'. The log ends with a Ctrl-C (^C) signal.

```
server1@server1:~/Desktop/A01/Phase1$ ./server 5000
server listening on port 5000
[*] connection from 10.176.76.207:61893
[+] Ram connected (fd=4)
[*] connection from 10.176.76.207:61894
[+] Keyaan connected (fd=5)
[relay] Ram -> Keyaan : Hi Keyaan!
[relay] Keyaan -> Ram : Hi Ram!
[relay] Ram -> Keyaan : \quit
[-] Ram disconnected
[-] Keyaan disconnected
[*] connection from 10.176.76.207:61916
[*] connection from 10.176.76.207:61918
[+] Ram connected (fd=4)
[*] connection from 10.176.76.207:61919
[+] Keyaan connected (fd=5)
[*] connection from 10.176.76.207:61917
[relay] Ram -> Keyaan : Hi Keyaan!
[relay] Keyaan -> Ram : Hi Ram!
[relay] Ram -> Keyaan : What are you doing?
^C
```

```
Connected as 'Keyaan'. Commands: @user msg | /chat user | /who | /quit
> [server] logged in
> /chat Ram
[now chatting with Ram]
>
[Ram] Hi Keyaan!
> Hi Ram!
>
[Ram] What are you doing?
>
```

(a) Client (Keyaan) chatting with Ram.

(b) Server log — every relayed message is printed in plaintext.

Figure 2: Phase 1 functional chat and server-side plaintext visibility.

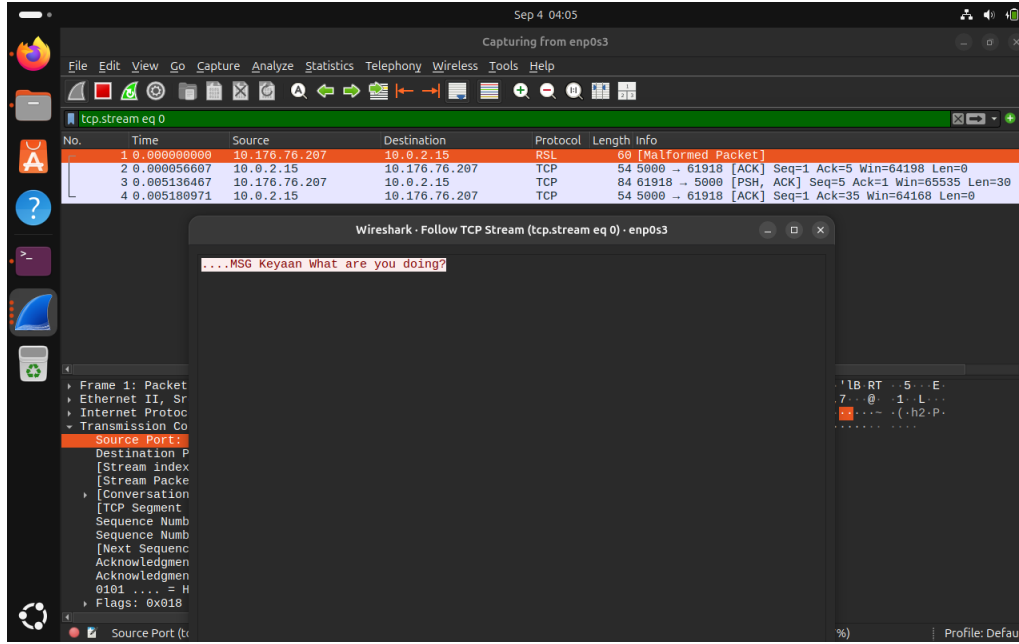


Figure 3: Phase 1 Wireshark capture (Follow TCP Stream): the message content is visible in plaintext on the network.

3 Phase 2 - Confidentiality via Diffie–Hellman + AES-GCM

3.1 Implementation

Diffie–Hellman from scratch. The exchange uses the standard **RFC 3526 MODP Group 14** (2048-bit) prime with generator $g = 2$. Modular exponentiation is our own square-and-multiply routine (`my_mod_exp`), which scans the exponent bit by bit and uses OpenSSL’s `BN_mod_mul` purely as an arbitrary-precision multiply:

```
void my_mod_exp(BIGNUM *result, const BIGNUM *base, const BIGNUM *exp,
               const BIGNUM *m, BN_CTX *ctx) {
    BN_one(result);          // result = 1
    BIGNUM *b = BN_new(); BN_mod(b, base, m, ctx);
    for (int i = 0; i < BN_num_bits(exp); i++) {
        if (BN_is_bit_set(exp, i)) BN_mod_mul(result, result, b, m, ctx);
        BN_mod_mul(b, b, b, m, ctx);      // b = b^2 mod m
    }
    BN_free(b);
}
```

Each client generates a fresh 2047-bit private exponent and computes $g^x \bmod p$; the two sides exchange public values and each computes $Y^x \bmod p$. Because each client connects and runs this independently, the $C_1 \leftrightarrow S$ and $C_2 \leftrightarrow S$ links have different keys.

Why hash the shared secret? The raw DH result is an element of the multiplicative group modulo a 2048-bit prime: it is 256 bytes long, is *not* uniformly distributed over those bytes, and carries algebraic structure. It is therefore unsuitable as an AES key directly. We pass it through **SHA-256**, which (i) compresses it to exactly the 32 bytes an AES-256 key needs, (ii) yields an (effectively) uniformly-distributed key from a non-uniform input, and (iii) destroys any exploitable structure. This is the key-derivation step `derive_key`.

Authenticated encryption. Every frame is sealed with **AES-256-GCM**: a fresh random 12-byte IV is generated per message and the wire blob is [IV (12)] [ciphertext] [tag (16)]. On receipt, GCM verifies the 128-bit tag before releasing any plaintext; a failed tag returns an error rather than corrupted data. Encryption begins at the LOGIN exchange, so even the login is protected. The server decrypts on the sender’s link and re-encrypts on the recipient’s link; consequently the server can still read chat at this phase (the limitation removed in Phase 4).

3.2 Verification and evidence

Fingerprint agreement. Figure 4 shows the client and server sides of the two exchanges. The $C_2 \leftrightarrow S$ link fingerprint 2c3e98b7e725578b printed by Keyaan’s client matches exactly the value the server prints for Keyaan, and the server separately shows Ram’s distinct link fingerprint 890d0ff64bed28d3 — confirming two independent, correctly-agreed keys.

```
client2@Client2:~/CS6008/Secure_Chat_Server_NetSec/Phase2$ ./client 10.176.76.20
6 5000 Keyaan
[dh] shared-secret fingerprint = 2c3e98b7e725578b
      (must match the server's printed fingerprint for this link)
Connected as 'Kyaan'. Commands: @user msg | /chat user | /who | /quit
> [server] logged in
> Hi Ram
[no partner selected]
> /chat Ram
[now chatting with Ram]
> Hi Ram
>
[Ram] Hi Keyaan
```

```
server1@server1:~/Desktop/A01/Phase2$ ./server 5000
server listening on port 5000
[*] connection from 10.176.76.207:62648
[dh] new client: shared-secret fingerprint = 890d0ff64bed28d3
[*] Ram connected (encrypted)
[*] connection from 10.176.76.207:62649
[dh] new client: shared-secret fingerprint = 2c3e98b7e725578b
[*] Keyaan connected (encrypted)
[relay] Keyaan -> Ram : Hi Ram
[relay] Ram -> Keyaan : Hi Keyaan
```

(a) Client (Kyaan): link fingerprint 2c3e98b7...

(b) Server: matching per-link fingerprints.

Figure 4: Phase 2 fingerprint agreement on both ends of each DH exchange.

Ciphertext on the wire. Repeating the Phase 1 capture on the now-encrypted traffic (Figure 5) shows only unreadable bytes in the TCP stream — the plaintext of Figure 3 is gone.

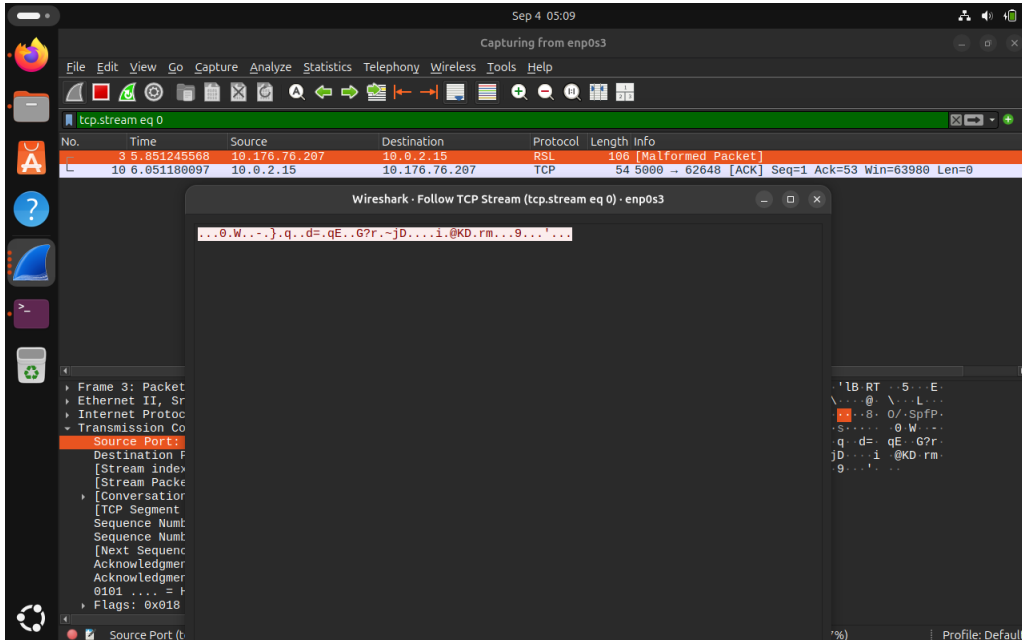


Figure 5: Phase 2 Wireshark capture: the same position that showed plaintext in Phase 1 now shows only AES-GCM ciphertext.

Tamper detection. Running the proxy in **-tamper** mode flips one byte of each frame forwarded to the victim. As shown in Figure 6, the victim’s client reports decryption/authentication

FAILED - message dropped instead of displaying corrupted text: GCM's integrity tag rejects the modification.

```
client2@Client2:~/CS6008/Secure_Chat_Server_NetSec/Phase2$ ./client 10.176.76.20
6 5001 Keyaan
[dh] shared-secret fingerprint = 3f152d0290f3f4e7
      (must match the server's printed fingerprint for this link)
Connected as 'Kyaan'. Commands: @user msg | /chat user | /who | /quit
>
[!] decryption/authentication FAILED - message dropped
>
[!] decryption/authentication FAILED - message dropped
```

(a) Victim client rejects the modified frame.

```
mallory@mallory:~/Downloads/CS6008/secure-chat/phase2$ ./mitm 5000 10.176.76.206 5000 --tamper
[MITM] listening on 5000, forwarding to 10.176.76.206:5000 (TAMPER MODE)
[MITM] victim connected from 10.176.76.207:62779
[MITM] key with VICTIM fingerprint = 3f152d0290f3f4e7
[MITM] key with SERVER fingerprint = 5bda689b2e2453d6
[MITM] both DH handshakes complete - now relaying (and reading) traffic
[CAPTURED victim->server] LOGIN Kyaan
[CAPTURED server->victim] OK logged in
[CAPTURED server->victim] MSG Ram Hi Kyaan
```

(b) Mallory (tamper mode) flipping a byte in transit.

Figure 6: Phase 2 tamper-detection test.

3.3 Attack task: Man-in-the-Middle

The proxy (mitm.cc) performs *two* independent DH exchanges — one with the victim (posing as the server) and one with the real server (posing as the client) — then decrypts, logs and re-encrypts each message. Plain DH authenticates nobody, so both endpoints happily key with whoever answered. Figure 7 shows Mallory reading the victim's login and messages in the clear while the victim believes the link is secure.

```
client2@Client2:~/CS6008/Secure_Chat_Server_NetSec/Phase2$ ./client 10.176.76.20
6 5001 Keyaan
[dh] shared-secret fingerprint = 5bda689b2e2453d6
      (must match the server's printed fingerprint for this link)
Connected as 'Kyaan'. Commands: @user msg | /chat user | /who | /quit
> [server] logged in
>
[Ram] Hi Kyaan
> /chat Ram
[now chatting with Ram]
> Hi Ram
```

(a) Victim (via Mallory on port 5001) — note the fingerprint 5bda689b...

```
mallory@mallory:~/Downloads/CS6008/secure-chat/phase2$ ./mitm 5000 10.176.76.206 5000
[MITM] listening on 5000, forwarding to 10.176.76.206:5000
[MITM] victim connected from 10.176.76.207:62705
[MITM] key with VICTIM fingerprint = 5bda689b2e2453d6
[MITM] key with SERVER fingerprint = bf244693a92c08f5
[MITM] both DH handshakes complete - now relaying (and reading) traffic
[CAPTURED victim->server] LOGIN Kyaan
[CAPTURED server->victim] OK logged in
[CAPTURED server->victim] MSG Ram Hi Kyaan
[CAPTURED victim->server] MSG Ram Hi Ram
```

(b) Mallory captures LOGIN and messages in plaintext.

Figure 7: Phase 2 MITM attack succeeding against unauthenticated DH.

4 Phase 3 - Server Authentication via PKI

4.1 Implementation

PKI setup. `setup_ca.sh` uses the OpenSSL CLI to create an RSA-2048 CA (self-signed root), generate the server key and CSR with CN=chatserver, and have the CA sign the CSR into `server.cert.pem`. It also creates an attacker self-signed certificate (same CN, *not* CA-signed) used to demonstrate the defence. Notably, the script sets a very short (one-minute) certificate validity; this is deliberate, so that the validity-period check can be demonstrated live (see below). The key OpenSSL steps are:

```
openssl genrsa -out ca.key.pem 2048
openssl req -x509 -new -nodes -key ca.key.pem -sha256 \
    -subj "/.../CN=CS6008-Root-CA" -out ca.cert.pem
openssl genrsa -out server.key.pem 2048
openssl req -new -key server.key.pem -subj "/.../CN=chatserver" -out server.csr.
pem
openssl x509 -req -in server.csr.pem -CA ca.cert.pem -CAkey ca.key.pem \
    -CAcreateserial -sha256 -out server.cert.pem
```

Handshake and validation. The Phase 3 handshake (before any application data) is:

1. $S \rightarrow C$: the server certificate (PEM).
2. C validates it against its trusted CA. `validate_cert` checks the signature with `X509_verify(cert, CA_pubkey)`, the validity window with `X509_cmp_time`, and the CN with `X509_NAME_get_text_by_NID`. Any failure aborts *before* a nonce, DH value or login is sent.
3. $C \rightarrow S$: a 32-byte random challenge nonce.
4. $S \rightarrow C$: the server DH public value.
5. $S \rightarrow C$: an RSA-SHA256 signature over $(nonce \parallel server_DH_pub)$, made with the server's *private* key.
6. C verifies that signature with the public key *from the certificate*. On failure it aborts.
7. $C \rightarrow S$: the client DH public value; both derive the key.

4.2 Verification and evidence

Legitimate flow. Figure 8 shows the client validating the certificate and proof-of-possession and then keying normally; the server log shows the matching per-client fingerprints (0bbd4af1a08a985c for Ram).

```
client1@Client1:~/CS6008/Secure_Chat_Server_NetSec/Phase3$ ./client 10.176.76.20
6 5000 Ram pki/ca.cert.pem
[pki] certificate OK: CA-signed, in validity period, CN=chatserver
[pki] proof-of-possession OK: server signed the challenge with its private key
[dh] shared-secret fingerprint = 0bbd4af1a08a985c
Connected as 'Ram'. Commands: @user msg | /chat user | /who | /quit
> [server] logged in
```

```
server@server1:~/Desktop/AB1/Phase3$ ./server 5000 ./pki/server.cert.pem ./pki/server.key.pem
server on port 5000
[*] connection from 10.176.76.207:63259
[dh] shared-secret fingerprint = 0bbd4af1a08a985c
[*] Ram connected
[*] connection from 10.176.76.207:63262
[dh] shared-secret fingerprint = 3970fbd2a33ed9f0
[*] Keyaan connected
```

(a) Client: certificate OK and proof-of-possession OK.
(b) Server: authenticated clients connect.

Figure 8: Phase 3 legitimate PKI flow.

MITM re-run now fails. Re-running the proxy against Phase 3 with a self-signed certificate is rejected at certificate validation: the client prints `certificate signature not from trusted CA` and refuses to continue (Figure 9). This is the direct contrast to Phase 2, where the identical network position succeeded (Figure 7).

```
client2@Client2:~/CS6008/Secure_Chat_Server_NetSec/Phase3$ ./client 10.176.76.20
6 5001 Keyaan pki/ca.cert.pem
[ABORT] certificate REJECTED: certificate signature not from trusted CA
Refusing to continue: server authentication failed.
```

```
[MITM/phase3] listening on 5000, mode=fake
[MITM/phase3] presenting cert './pki/attacker_fake.cert.pem', signing with './pki/attacker_fake.key.pem'
[MITM/phase3] victim connected from 10.176.76.207:63268
[MITM/phase3] [2023-] victim rejected the certificate and hung up.
-> our cert was not signed by the trusted CA. Attack failed.
```

(a) Victim aborts — forged certificate rejected.
(b) Mallory observes the victim reject and hang up.

Figure 9: Phase 3 MITM re-attempt is defeated by certificate validation.

Validity-period check. Because the issued certificate is valid for only one minute, connecting after it expires exercises validation check (b): the client aborts with `certificate has expired` (Figure 10). This demonstrates that expiry — not just a bad signature — is enforced.

```
client2@Client2:~/CS6008/Secure_Chat_Server_NetSec/Phase3$ ./client 10.176.76.20
6 5000 Keyaan pki/ca.cert.pem
[ABORT] certificate REJECTED: certificate has expired
Refusing to continue: server authentication failed.
```

Figure 10: Phase 3 validity check: an expired certificate is rejected and the connection aborted.

5 Phase 4 — End-to-End Encryption Between Clients

5.1 Implementation

/e2e bob runs a *second* Diffie–Hellman directly between the two clients, carried as ordinary chat payloads that the server relays verbatim. The three wire tags are:

__E2E_INIT__<hexpub>	initiator’s DH public value
__E2E_ACK__<hexpub>	responder’s DH public value (completes the exchange)
__E2E_MSG__<hexblob>	an end-to-end AES-GCM encrypted chat message

The receiver dispatches on the tag prefix, so a handshake message is never shown as chat and a chat message is never mistaken for a handshake. Once the end-to-end key is derived (SHA-256 of that second DH secret), a chat message is first sealed with AES-GCM under the $C_1 \leftrightarrow C_2$ key and only then sent inside the already-encrypted Phase 3 link — a genuine inner layer, not a replacement.

The server does not change. The Phase 3, 4 and 5 servers are the same. The server sees the public tags and the routing metadata (who is messaging whom) but only the DH *public* values pass through it, from which it cannot compute the shared key, and the *content* inside __E2E_MSG__ is opaque ciphertext.

5.2 Verification and evidence

Fingerprint agreement without the server. Ram (initiator) and Keyaan (responder) independently derive the same end-to-end fingerprint d7b84202452b023d (Figure 11), even though their client–server link fingerprints (439760... and b16bfb...) differ and the server was not involved in the end-to-end exchange.

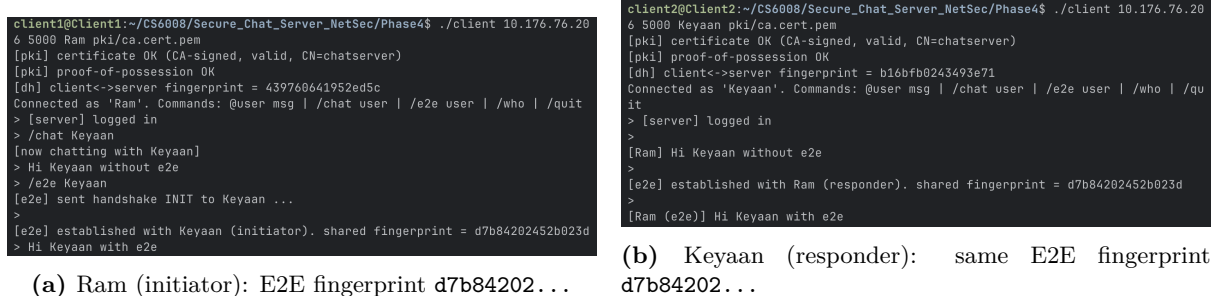


Figure 11: Phase 4 end-to-end key agreement, computed by the two clients without the server.

Server blindness. Figure 12 is the server relay log. The message sent *before* /e2e (Hi Keyaan without e2e) is readable, matching Phase 2/3 behaviour; after the session is established the log contains only __E2E_INIT__, __E2E_ACK__ and __E2E_MSG__ followed by opaque hex — the server cannot read Hi Keyaan with e2e.

6 Phase 5 — Forward Secrecy

6.1 Implementation

Timed rekey with epochs. A timer thread renegotiates the key every 60 seconds (the interval is overridable via the E2E_REKEY_SECONDS environment variable, used only to speed up demonstrations). Each key carries an integer *epoch*, and messages are stamped with the epoch

Figure 12: Phase 4 server log: plaintext before /e2e, opaque __E2E_INIT__/_E2E_ACK__/_E2E_MSG__ data afterwards.

that encrypted them (__E2E_MSG__<epoch>:<hex>). Every rotation performs a fresh ephemeral Diffie–Hellman, so each epoch’s key is independent of all others.

No disruption; disciplined key discard. Each client keeps the *current* key and the *immediately previous* key. A message that was in flight across a rotation is stamped with the previous epoch and is still decryptable with the retained previous key, so the live session is never broken. The previous key is used only for decrypting such stragglers — never to encrypt new messages — and the key before it is overwritten (discarded) at the next rotation, honouring “the old key must be discarded and never reused once the new key is confirmed”.

Collision avoidance by construction. Both clients run a rekey timer, but only the client with the lexicographically *smaller* username actually initiates a rotation (`am_initiator(peer) = (g_user < peer)`); the other merely responds. Because exactly one side ever initiates, two competing exchanges cannot occur, so the sides can never end up on different, disagreeing keys. This *prevents* the simultaneous-trigger problem rather than trying to resolve it after the fact, which is simpler and less error-prone. Two convenience commands, `/rekey` (force a rotation) and `/keyage` (print epoch, fingerprint and age), aid testing.

6.2 Verification and evidence

Rotations agree on both sides. Figure 13 shows both clients across the initial establishment and two automatic rotations. The fingerprint changes every epoch — 03dacc07854586df (epoch 1), e4e455c860e830a1 (epoch 2), daeb77e6ab85356b (epoch 3) — and both clients print the *same* fingerprint for each epoch, with timestamps about 60 seconds apart. The roles illustrate the collision-avoidance rule nicely: Ram started the session (so it is the *initiator* of epoch 1), but because “Keyaan” < “Ram”, Keyaan drives every *rotation* — so Ram logs the rotations as *responder* while Keyaan logs them as *initiator*.

Post-rotation delivery and server blindness. The message [Keyaan (e2e/e3)] Hi Ram in Figure 13(a) is a message encrypted under the epoch-3 key and correctly decrypted after two rotations, proving the rotation did not break the session. The server relay log (Figure 14) shows every rotation handshake and message as opaque, epoch-stamped data (__E2E_MSG__1:..., __E2E_INIT__2:..., ...): the server relays rekeys and traffic without ever seeing content.

```

client1@Client1:~/CS6808/Secure_Chat_Server_MetSec/Phase5$ ./client 10.176.76.206 5000 Ram pki/ca.cert.pem
[pm] certificate OK (CA-signed, valid, CN=chatserver)
[pm] proof-of-possession OK
[dm] client->server fingerprint = 386d6b32blada5c6
Connected as 'Ram'. Commands: @user msg | /chat user | /e2e user | /rekey | /keyage | /who | /quit
> [server] logged in
> /e2e Keyaan
[e2e] initiating with Keyaan ...
>
[rekey] established (initiator) epoch=1 key-fingerprint=03dacc07854586df at 11:16:18
> /chat Keyaan
[now chatting with Keyaan]
> Hi Keyaan
>
[Keyaan (e2e/e1)] Hi Ram with e2e
>
[rekey] rotated (responder) epoch=2 key-fingerprint=e4e455c8e0e830a1 at 11:17:08
> Hi Keyaan
>
[rekey] rotated (responder) epoch=3 key-fingerprint=daeb77e6ab85356b at 11:18:08
>
[Keyaan (e2e/e3)] Hi Ram

```

(a) Ram: epoch 1 initiator, then responder for rotations.

```

client2@Client2:~/CS6808/Secure_Chat_Server_MetSec/Phase5$ ./client 10.176.76.206 5000 Keyaan pki/ca.cert.pem
[pm] certificate OK (CA-signed, valid, CN=chatserver)
[pm] proof-of-possession OK
[dm] client->server fingerprint = 28f297d683fe10bb
Connected as 'Kyaan'. Commands: @user msg | /chat user | /e2e user | /rekey | /keyage | /who | /quit
> [server] logged in
>
[rekey] established (responder) epoch=1 key-fingerprint=03dacc07854586df at 11:15:41
>
[Ram (e2e/e1)] Hi Keyaan
> Hi Ram with e2e
>
[rekey] rotated (initiator) epoch=2 key-fingerprint=e4e455c8e0e830a1 at 11:16:32
>
[Ram (e2e/e2)] Hi Keyaan
>
[rekey] rotated (initiator) epoch=3 key-fingerprint=daeb77e6ab85356b at 11:17:32
> Hi Ram

```

(b) Keyaan: epoch 1 responder, then initiator for rotations.

Figure 13: Phase 5 rotations: fingerprints change each epoch and agree across both clients; timestamps are ≈ 60 s apart.

```

server1@server1:~/Desktop/AB1/Phase5$ ./server 5000 ./pki/server.cert.pem ./pki/server.key.pem
server on port 5000
[*] connection from 10.176.76.207:64878
[dh] shared-secret fingerprint = 386d6b32blada5c6
[*] Ram connected
[*] connection from 10.176.76.207:64874
[dh] shared-secret fingerprint = 28f297d683fe10bb
[*] Keyaan connected
[relay] Ram -> Keyaan : __E2E_INIT__1:83d84a162d6207d3fd039961a223522abadd13f9ab4f363d811647062945831eab9c727e916f595a2a7c9aa1d6e6646c752f4f3583b9e40dc1097a
571bb0f42b0501a1d53b34edc13723463ba57176f58314dd3682b4c48eeacdd110fb765b69e0ad1cb8ec334c6e5802a47ae8993388a1f0840762d67cddbbce948da93d2889081720bb0967f852d2
2b0c1d3b02f3aedcc3c75a4d78fa1574eb448aa90791851b431e15d57d913393664324dc211740112c549fab6e03000168a9b607c88e46a2cec240c6efc002ab0865e7577c1573485cba95066e5
58c04a9e245dfcc22a4e511ab3b8cb0caac80133d6fa5fe19a2a16822959e3a0bf956cd0097a639b0
[relay] Keyaan -> Ram : __E2E_ACK__1:8fc99b84c9fa3f2985889858b2a90399ace92112a20f9fc0eb9eb93a59a653d7f5aa4443cfa2dcf4f29f7b1af25e8101f39f4ff7b3f1f0516da44038
f5a4b36b73323582dd3e2fcaa56b8e1f1835d641aeca3a625e96b8519e2c6f0d95fb9c958236028f4282ac9f12ccf57726c2a5f25d715f3d5788d7f20e90c923085cb45afbe2c8b4393cc4c87f7b4
9e62579f249c895b28369df2b69e51ac891d440971c4128c4a4df95c75035e11ec3efd7c1d70eb9a6605ef9f0bd0464a6bb275ba04a4e627998411e46fae4f6ee1833eb096c737b43053a3b56d2
450685b4009fa9d0b8c4a86006407e1ebb27298a556faa1a6ed6223da1802914f365f1482b647e3ae
[relay] Ram -> Keyaan : __E2E_MSG__1:e55cc4961738995d4627321f120cd6449c35b42749e9ec77ae081d79e1c89f16567b4e225
[relay] Keyaan -> Ram : __E2E_MSG__1:d01ed10b171d55d16e462e10b9d06badd3689446184c4d16652ec73c9258e10e6c6b5ecb46d7c1d0f0beed
[relay] Keyaan -> Ram : __E2E_INIT__2:58ba724565f6a107fe2b6a61a3ec9e103c6e102d9f3fadcd793b969acbb5edf8f5fff5d0e4757c0861e2ecc75afee3dd9782751b3ccde3c834757b
ebfbaec3762e0b87907e9211c1f4fcab5751ece24bd39bca5915e4f4e24fdce4c3880c7f063ec4e25972046cf9ac46325b337bc154089595664e5611df82c3e19df8822ab871dc8be40c59601b809
97490bb0b9f2ad0f19d11a6ace20e70892e637f4db050499b37e1c08f3fa5976f4ccc91275dcffce045cc180988f4ee3367cf8fa95e521e19967ce5cd50c0a1904b4e086471c7d861f19581c9c
68136583bea03937d4f1908b332040070e919397711bbeafa602a05b1c9089ce8a8701991c35f75966
[relay] Ram -> Keyaan : __E2E_ACK__2:05f566d0ec97be1abc2499513407c3f6fd5772cb404d3016545c43aa690aa869525b7389d4ddfb68a8d151f14e0b2b2d3e9afe60389eae99972753212
3d03ef2dbd7e46c363224d5b3d23dccc2a35aa12ca092744adcc1219a0d133fa761fe70f6286f322fbf9cc226bc7f645818bfcde95a39d5b1c501a2a90cdd8757b1a87165ea29fb40370cef295587b
1de03e559d88e290cb1efc70e9466f5e1fdd44a3bb4a57fb8fded7b4fffc3e46496adbb6e9e6a2ce95cd22162e479034a60fa2ca2bf18363f07bb3f61b046be8214a6d896fe9c3669fca17b7df
6ddbcad1e0633cb55e04e90e3537f881993d70c4582b8f1c0ba54c340690bae6708b8e0dbd1f86e7
[relay] Ram -> Keyaan : __E2E_MSG__2:588117beec7740b87347f360de146039734e81a84e2324dc91ec805d69affa14bec7fd06
[relay] Keyaan -> Ram : __E2E_INIT__3:7e47355288d7027d846ce733e3b0778d23bce622f9b52a5dc4073c6a125886aca77a04883244fa9355489aa8875b9e6b627fc56c34ee09898d89d
75efale86347ead23f5645b5db3e9b09f2b19cee98806505df242fbd0b11c3d2531871373a53049f3cabde10d33c4fc571bbe3b282af6d32e7efdae4412471f510a54e4aa144e74468d42fb8b901
5b9a73bdc95b6498bebb9a2e0e208c497223053afal7d259d2025280b9af26e9d210dcce9ef4d75729896c29a58621bc7a3fccc78c85492be6b12c1ccc9f723e205322366126dd8a87f6162a23a
7dc514fc183bf1be6e0839c392a8d6f59d823755c3440971fd5642b520a6c72060457dd34bcb689c996a
[relay] Ram -> Keyaan : __E2E_ACK__3:17cd619e0bf17f5b2112a00a10062f220552e1465141c50824bf4f1df31a73da86dbf8e47a3ae888122e619ae7da3dd562b69db3a20363f9d697bde
8f6f9c4f758614470dcff36ab8a8b5b1fcfd08c942d9291af44c6e53c3947d804e92a5156694f04a0b3ca7e2430f0bf0da8f1a0b2e250e9f3f8e62c0e50b35a5b1f0658619a48a75b72785e578ead
15e587beafe74fb83c5d2f8de1718calcc74f8a1632c73d2d8e865af6d0f060beeb8c7dc8c8a6da5f6cad082699773933b1a6b353ecf6a7497c7d2cd4b12d0a9256eef983e8e483d5140fa7a
fe6dc7ede65fb14689f0421b3bec9249a2075f9be64dc10d3e5acbdd349b36b08a1fd257b721534
[relay] Keyaan -> Ram : __E2E_MSG__3:8b17b863c6de0f0f50f534ebfeff82f994a7bc76f31a38b6f96b9e2a99d27cfc90ce51

```

Figure 14: Phase 5 server log: rotation handshakes and messages relayed as opaque, epoch-stamped data.

7 Submission and Build

7.1 Repository structure

```
Phase 1: baseline plaintext chat (framing, relay server, client CLI)
Phase 2: DH-from-scratch + AES-256-GCM per link; MITM proxy + tamper demo
Phase 3: PKI server auth (CA-signed cert + proof-of-possession); MITM defeated
Phase 4: end-to-end encryption between clients; server relays opaquely
Phase 5: forward secrecy via 60s ephemeral-DH key rotation with epochs
```

7.2 Build and run (per phase)

```
# Phase 1
cd Phase1 && make
./server 5000                                # Server VM
./client 10.176.76.206 5000 Ram                # Client VM
# Phase 2 (adds mitm)
cd Phase2 && make
./server 5000
./client 10.176.76.206 5000 Ram
./mitm 5000 10.176.76.206 5000 [--tamper]      # Mallory VM
# Phases 3-5 (need certificates first)
cd Phase3 && make && ./setup_ca.sh
./server 5000 pki/server.cert.pem pki/server.key.pem
./client 10.176.76.206 5000 Ram pki/ca.cert.pem
# Phase 3 attacks:
./mitm 5001 fake pki/attacker_fake.cert.pem pki/attacker_fake.key.pem
./mitm 5001 stolen pki/server.cert.pem pki/attacker_fake.key.pem
# Phase 5 fast-rotation demo (default is 60s):
E2E_REKEY_SECONDS=60 ./client 10.176.76.206 5000 Ram pki/ca.cert.pem
```

A Generative AI Usage

LLM assistance was used to help scaffold portions of the implementation/setting up and to draft this report. A log of the prompts used is included here as required by the course policy.

1. prompts related to port forwarding in virtual box to help us communicate between vms of different machines
2. prompts related AES-GCM, certificates (as we have never used this before)
3. prompts to style and paraphrase this report